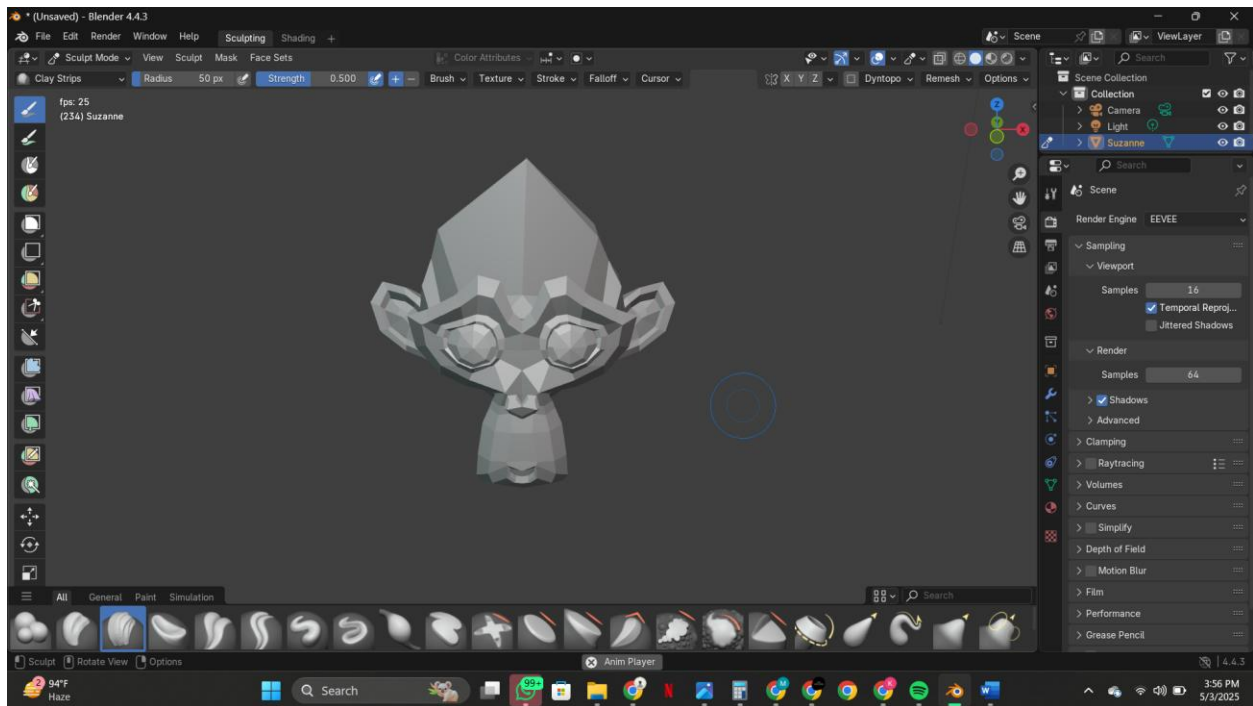
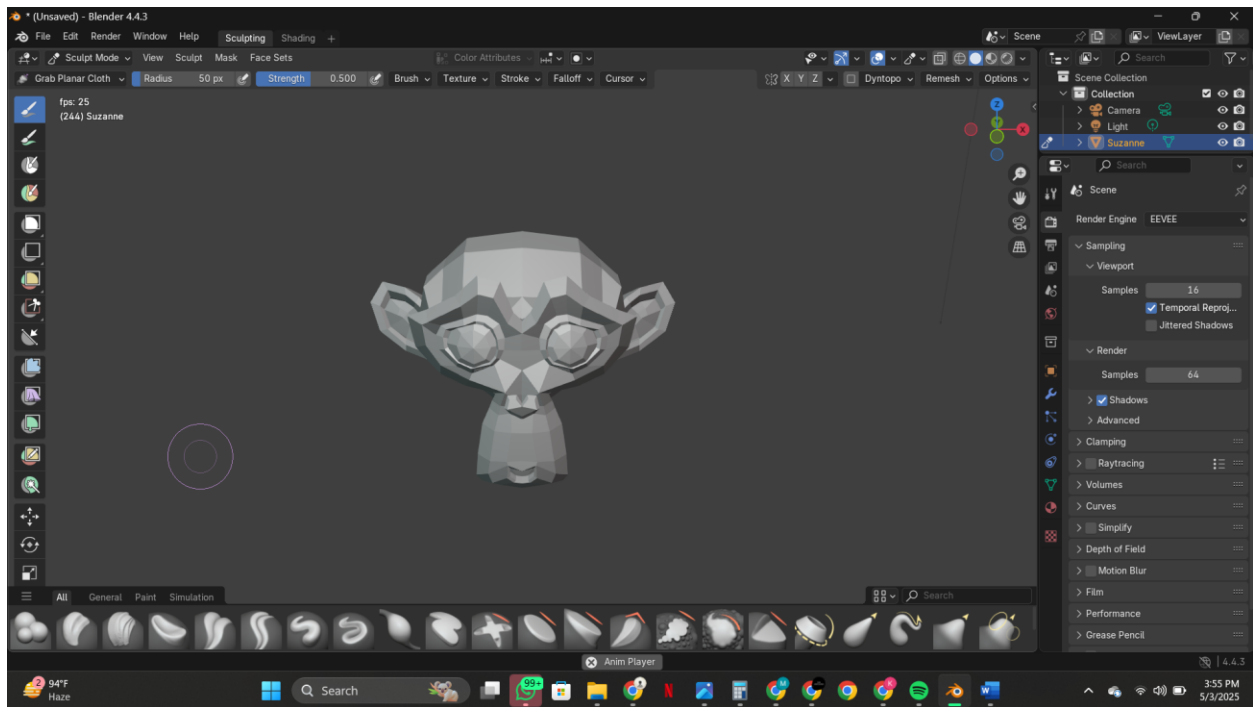
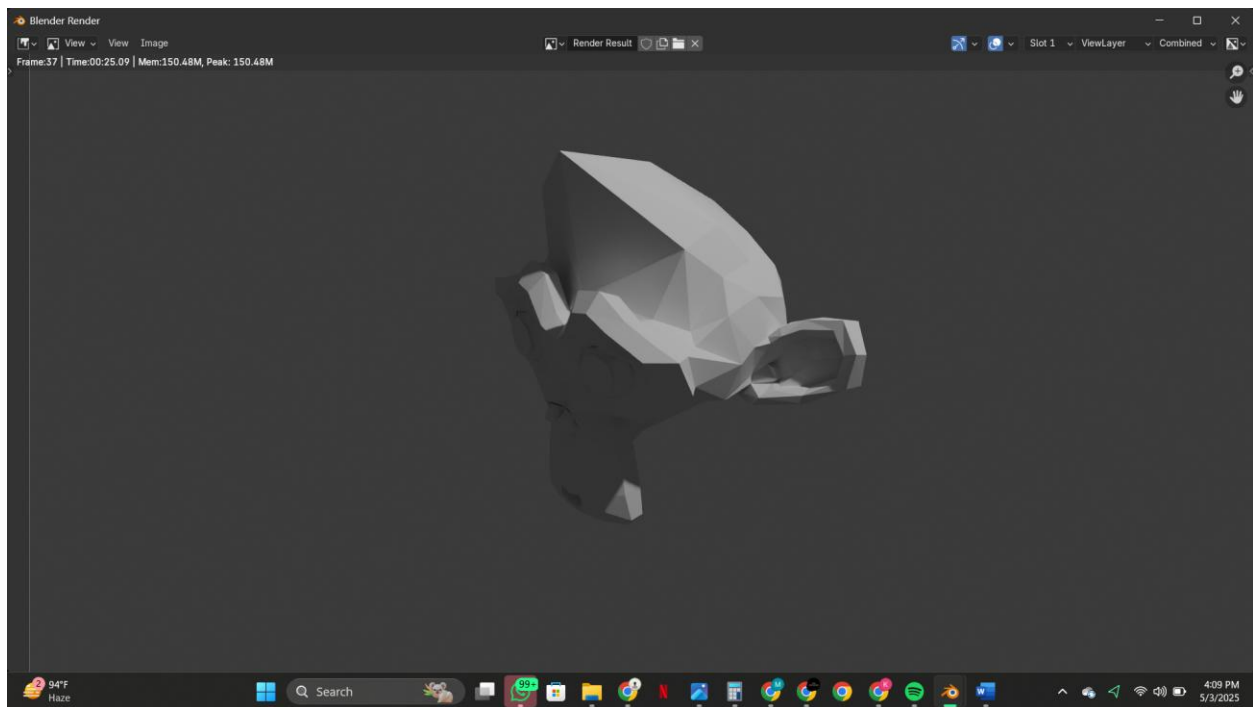
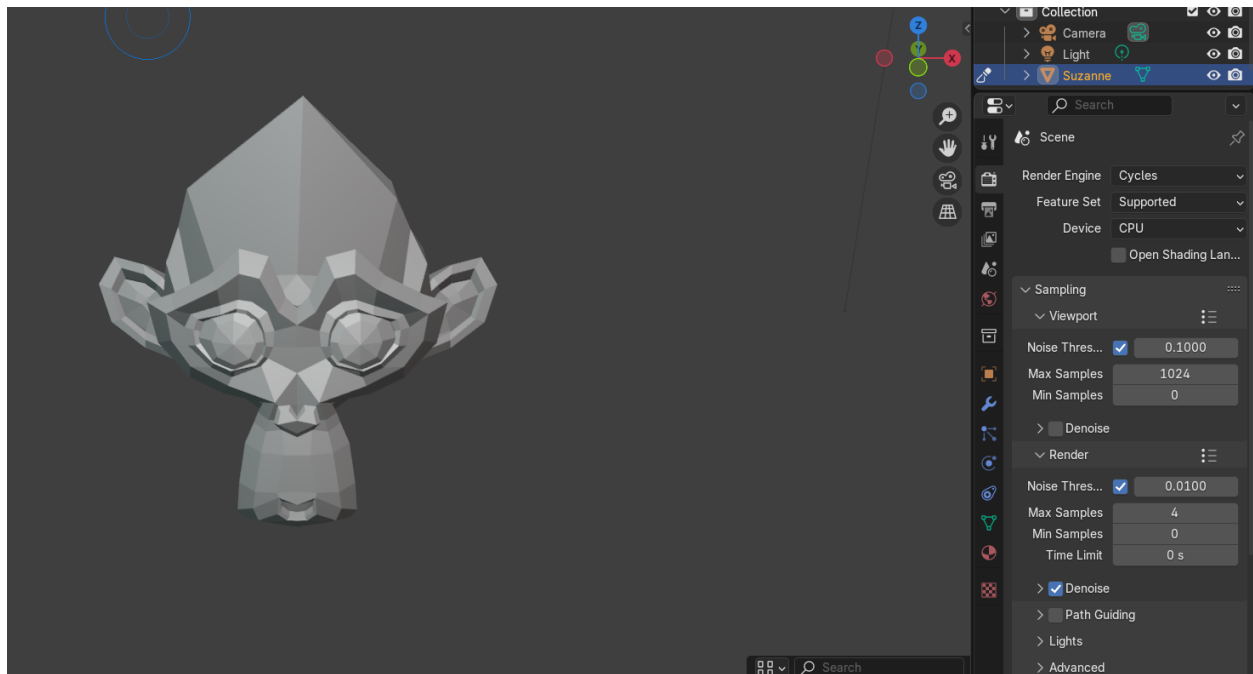


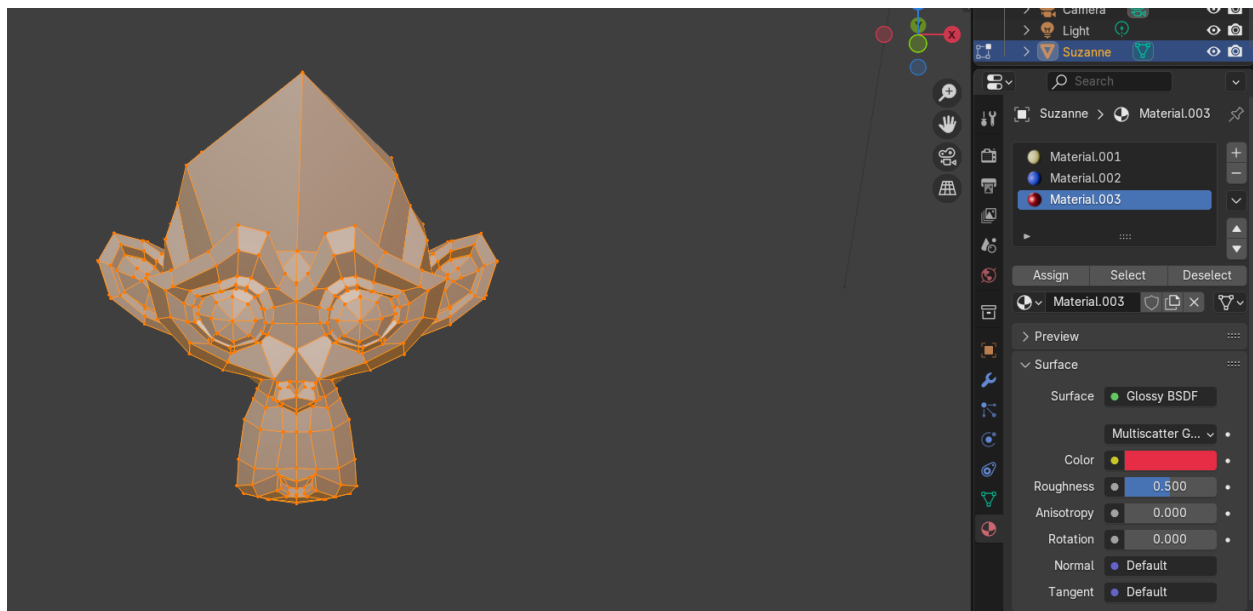
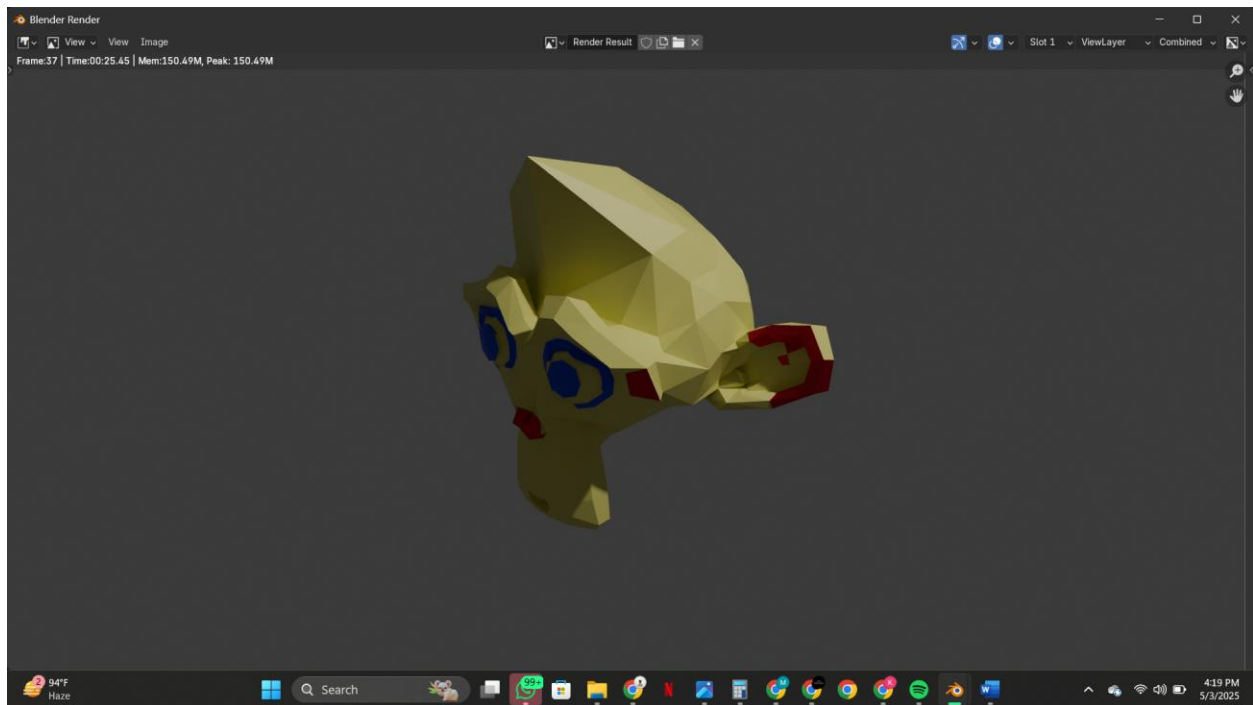
Question1:



Task 2:

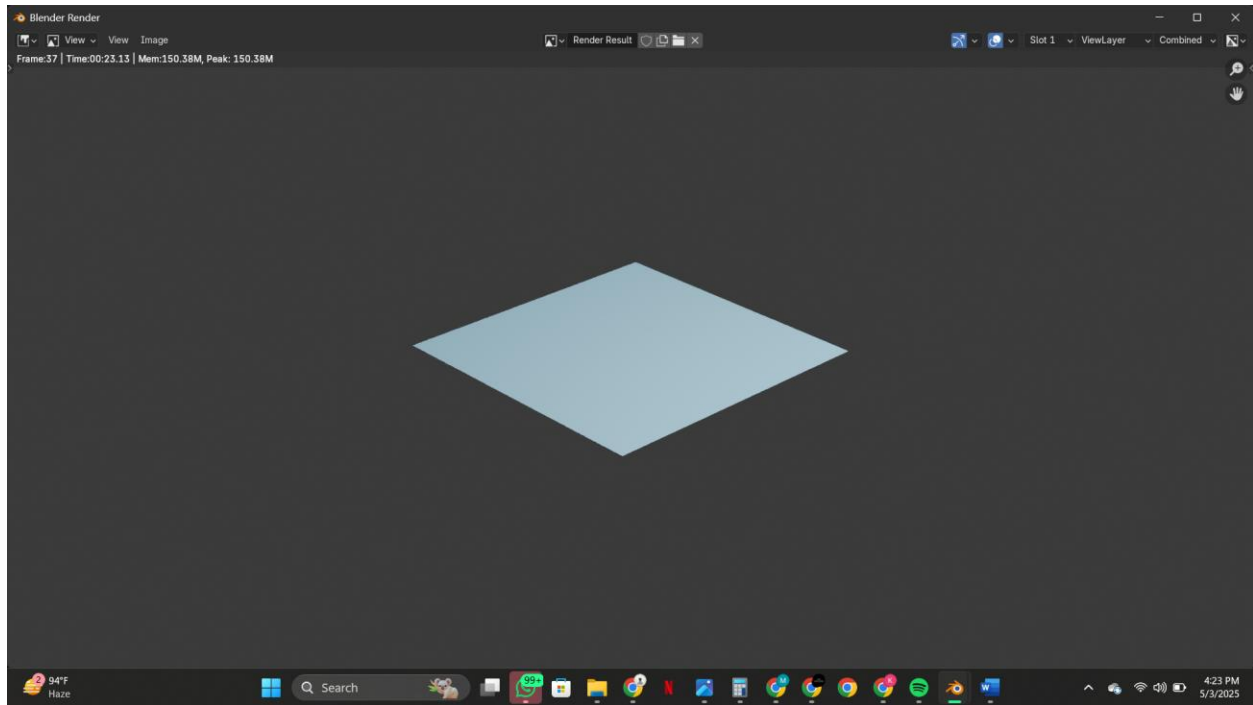


Task 3:

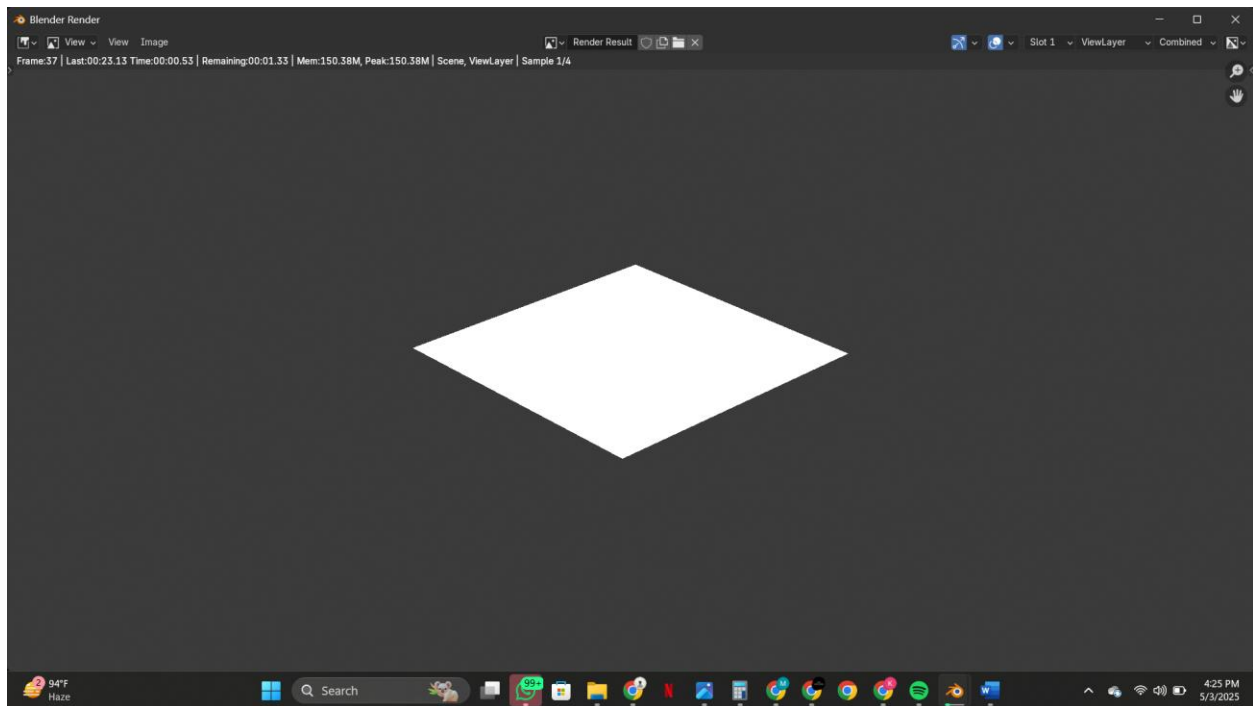


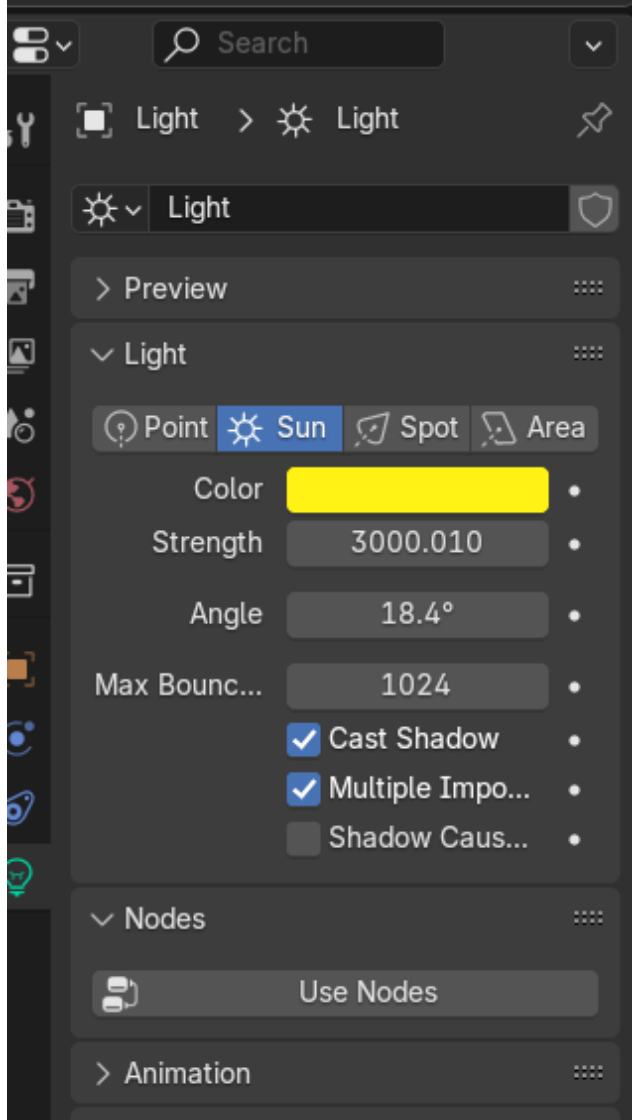
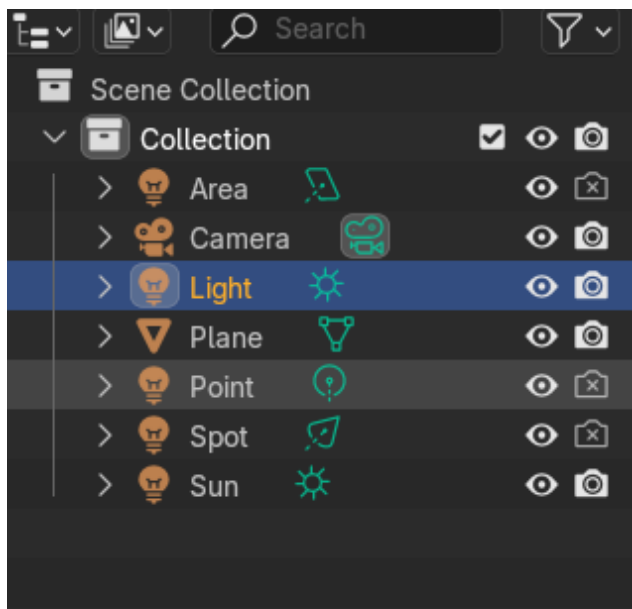
Task 4:

Point:

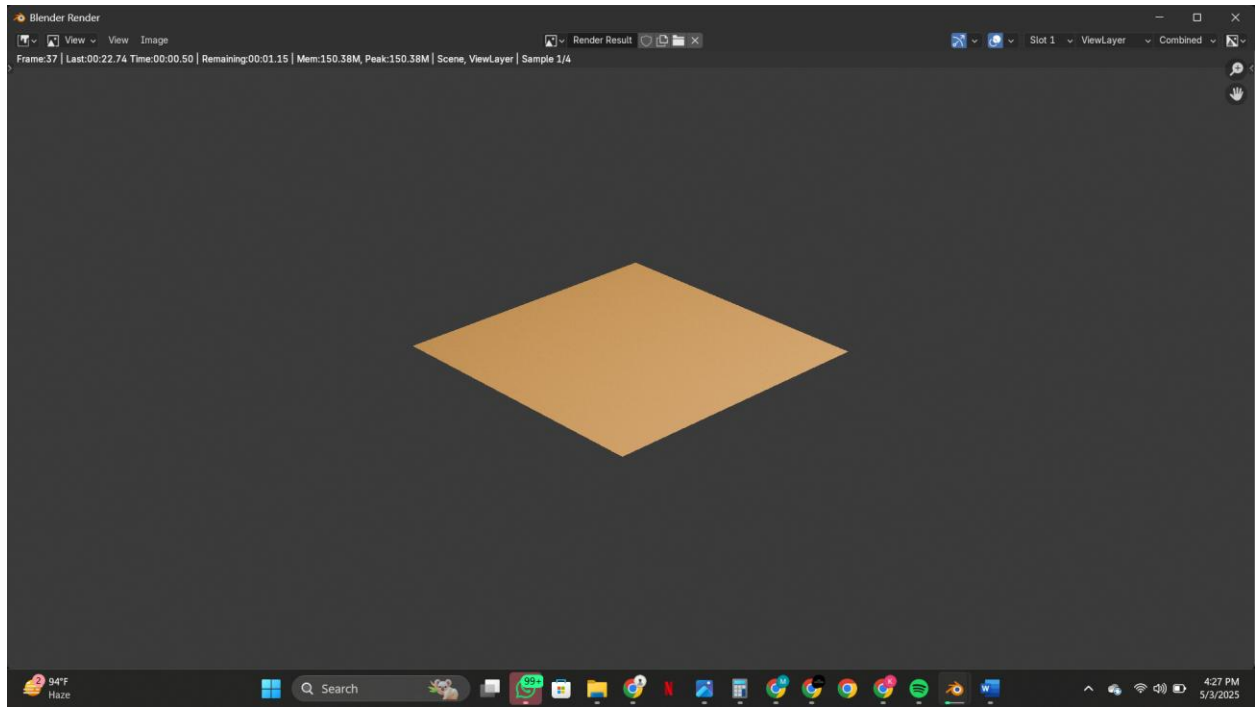


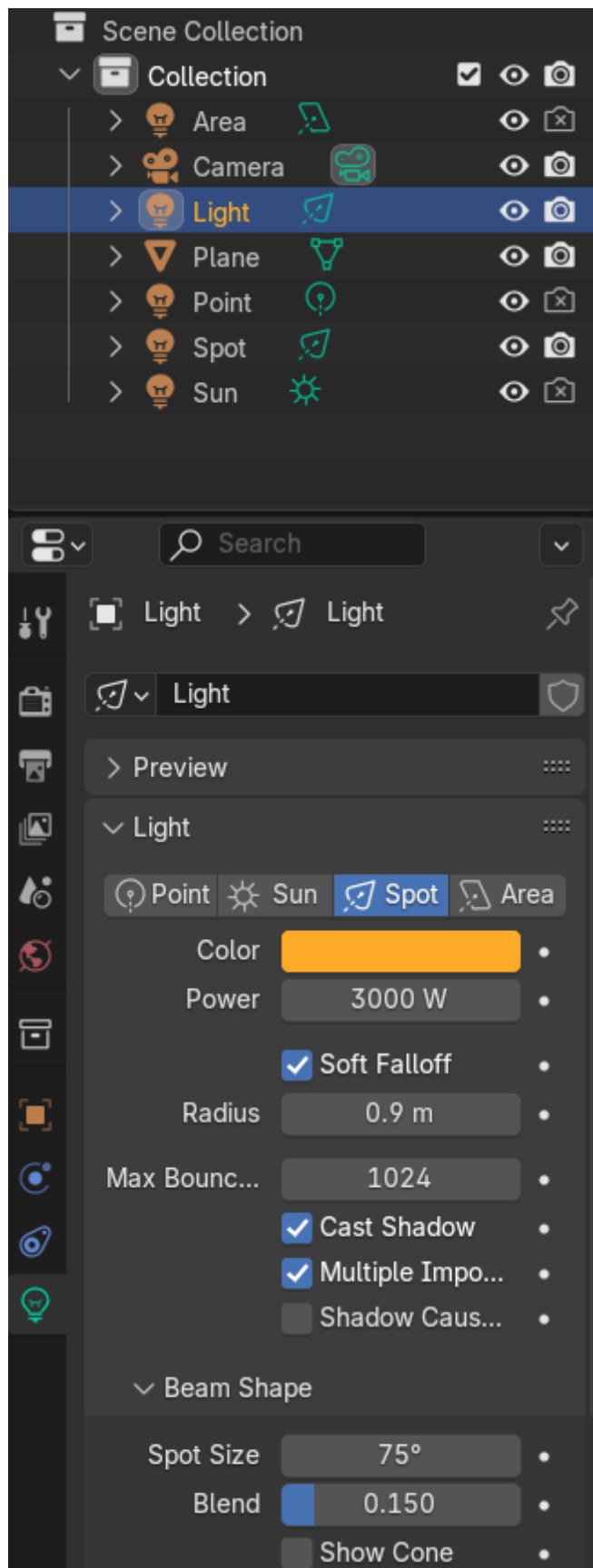
Sun:



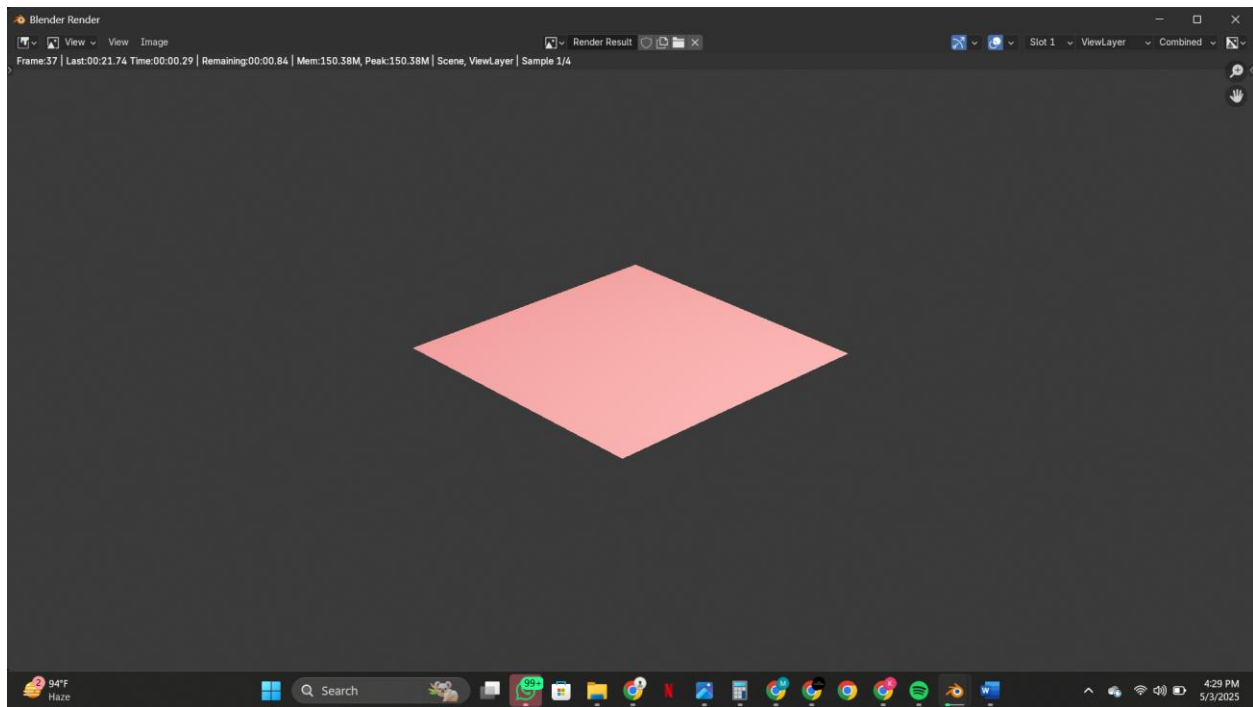


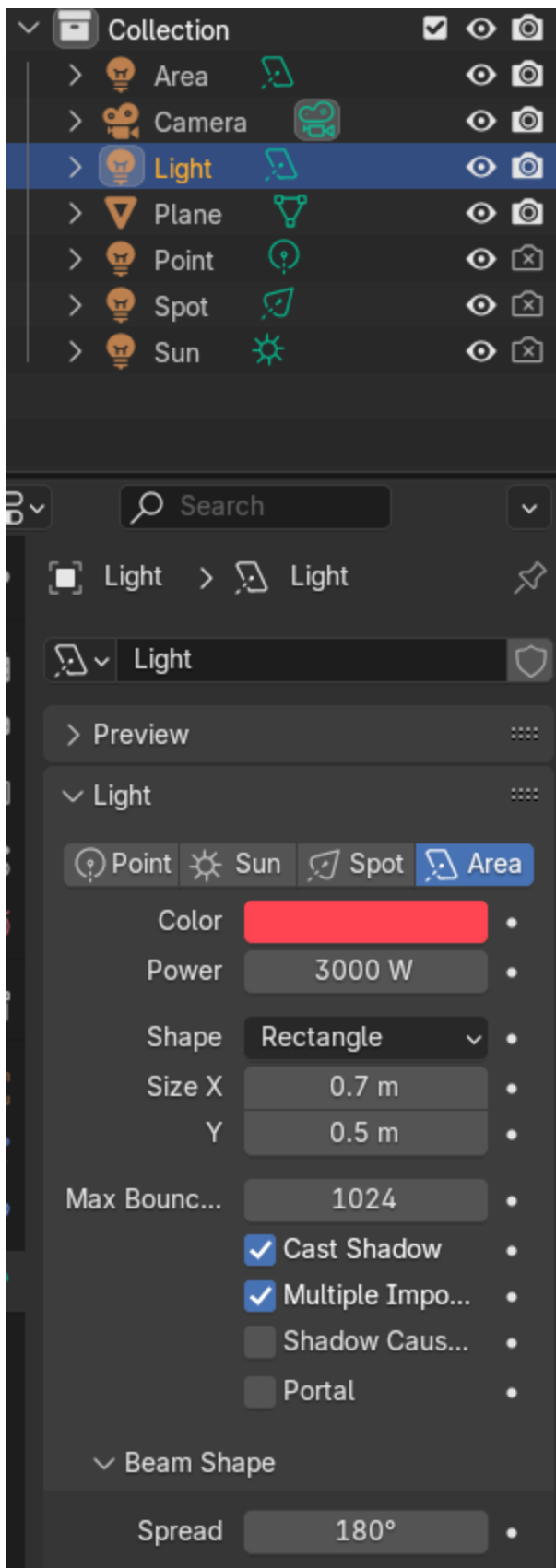
Spot:



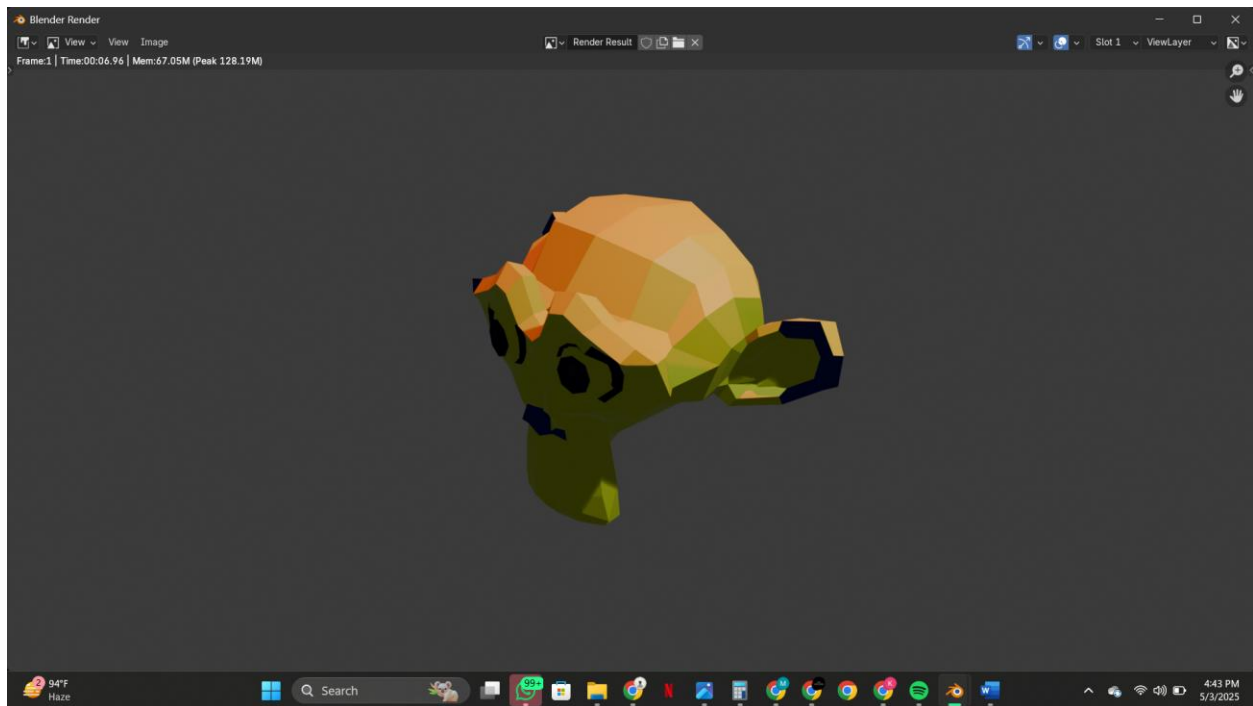
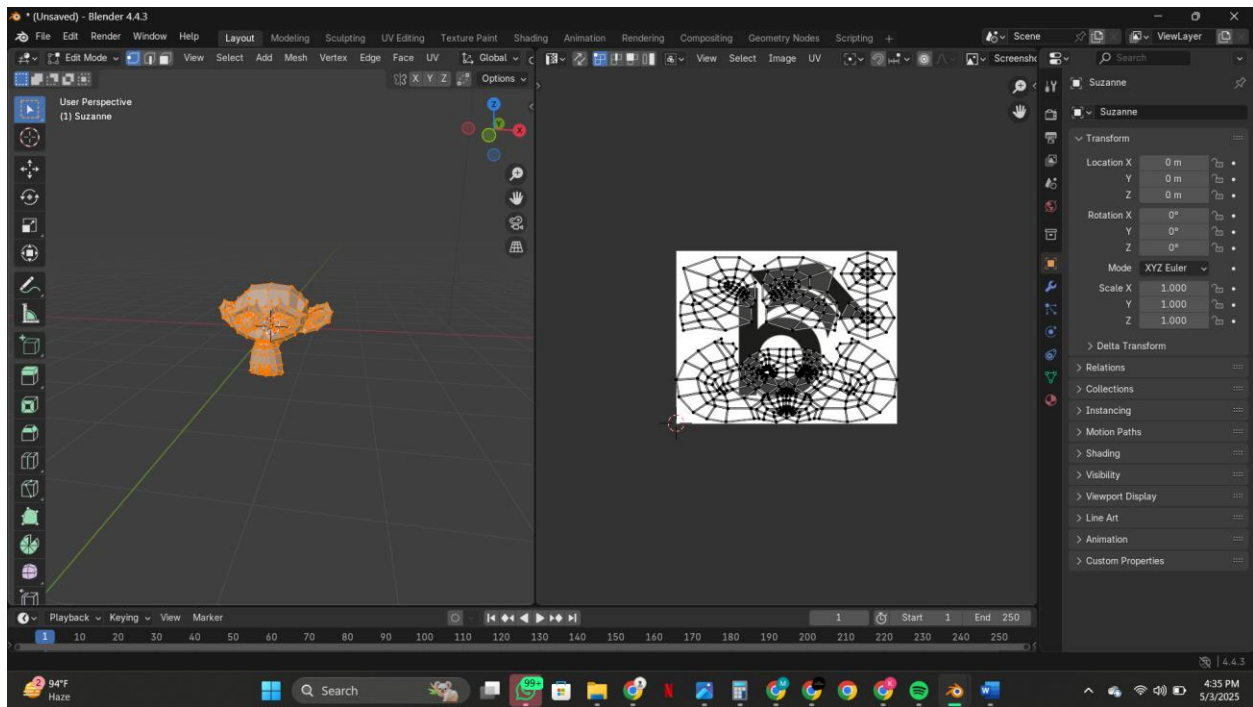


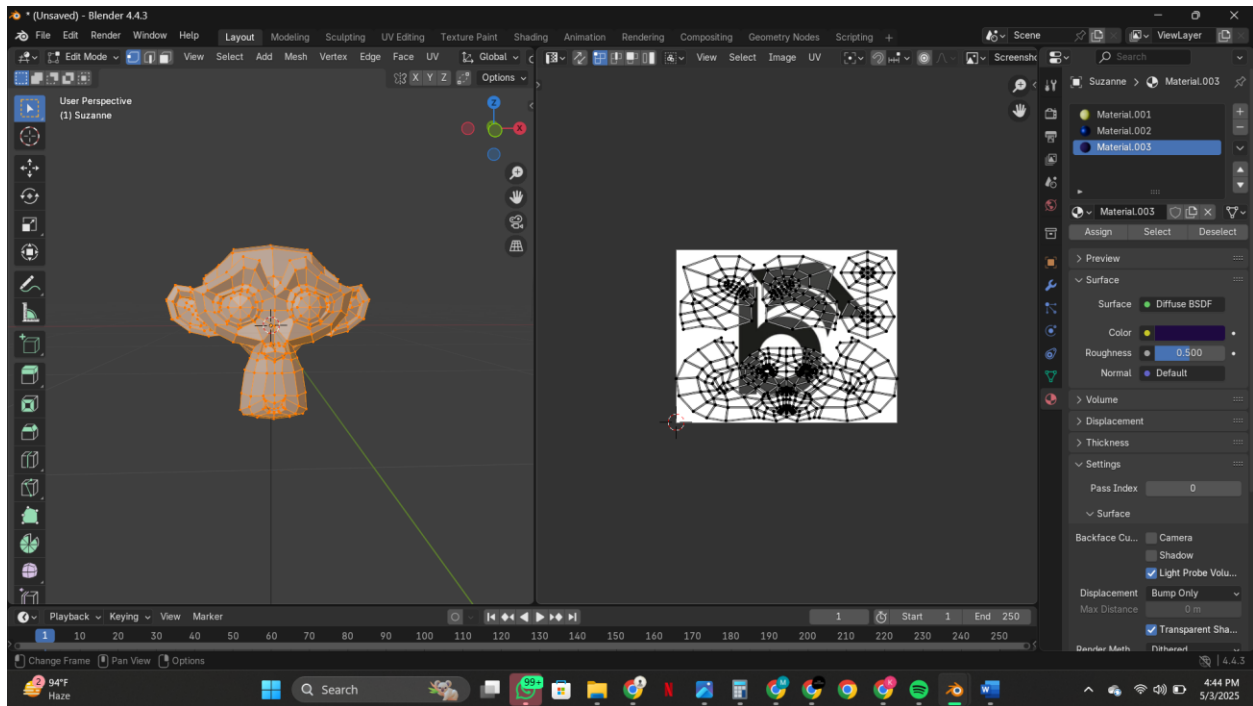
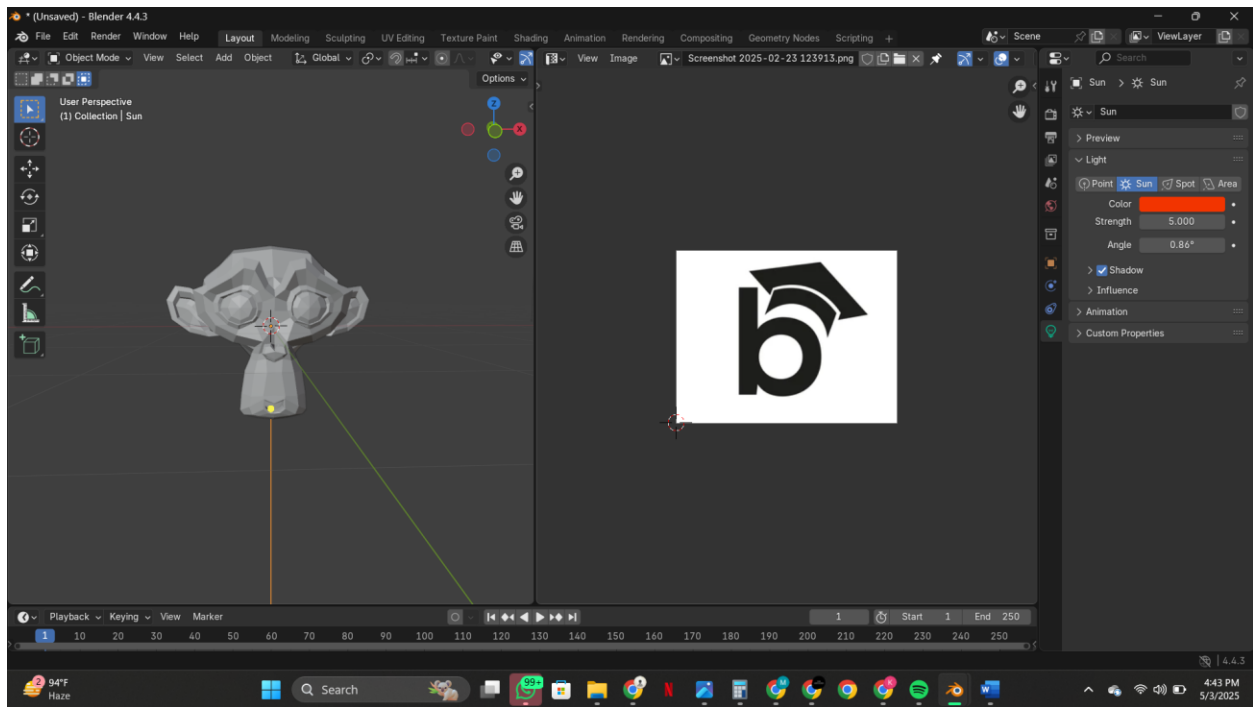
Area:





Task 5:





Task 6:

Installing Libraries :

```

C:\Users\lenovo\Desktop\CG Task 6> git clone https://github.com/microsoft/vcpkg.git C:\vcpkg
>> cd C:\vcpkg
>> .\bootstrap-vcpkg.bat
>> .\vcpkg integrate install
>> .\vcpkg install glm:x64-windows glm:x64-windows stb:x64-windows
>>
Cloning into 'C:\vcpkg'...
remote: Enumerating objects: 273494, done.
remote: Counting objects: 100% (654/654), done.
remote: Compressing objects: 100% (401/401), done.
Receiving objects: 100% (273494/273494), 85.06 MiB | 422.00 KiB/s, done.
remote: Total 273494 (delta 543), reused 253 (delta 253), pack-reused 272840 (from 4)
Resolving deltas: 100% (181546/181546), done.
Updating files: 100% (12363/12363), done.
Downloading https://github.com/microsoft/vcpkg-tool/releases/download/2025-04-16/vcpkg.exe -> C:\vcpkg\vcpkg.exe... done.
Validating signature... done.

vcpkg package management program version 2025-04-16-f9b6c6917b23c1ccf16c1a9f015ebabf8f615045

See LICENSE.txt for license information.
Telemetry
-----
vcpkg collects usage data in order to help us improve your experience.
The data collected by Microsoft is anonymous.
You can opt-out of telemetry by re-running the bootstrap-vcpkg script with -disableMetrics,
passing --disable-metrics to vcpkg on the command line,
or by setting the VCPKG_DISABLE_METRICS environment variable.

Read more about vcpkg telemetry at docs/about/privacy.md
Applied user-wide integration for this vcpkg root.

```

Vertex Shader (shaders/basic.vert):

```

#version 330 core

layout(location = 0) in vec3 aPos;

layout(location = 1) in vec3 aNormal;

layout(location = 2) in vec2 aTexCoord;

out vec3 FragPos;

out vec3 Normal;

out vec2 TexCoord;

uniform mat4 model;

uniform mat4 view;

uniform mat4 projection;

void main() {

    FragPos = vec3(model * vec4(aPos, 1.0));

    Normal = mat3(transpose(inverse(model))) * aNormal;

    TexCoord = aTexCoord;

    gl_Position = projection * view * vec4(FragPos, 1.0);

}

```

Fragment Shader (shaders/basic.frag)

```

#version 330 core

out vec4 FragColor;

in vec3 FragPos;
in vec3 Normal;

```

```

in vec2 TexCoord;

uniform sampler2D texture1;
uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 viewPos;

void main() {
    // Ambient
    float ambientStrength = 0.1;
    vec3 ambient = ambientStrength * lightColor;

    // Diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * lightColor;

    // Combine with texture
    vec3 texColor = texture(texture1, TexCoord).rgb;
    vec3 result = (ambient + diffuse) * texColor;
    FragColor = vec4(result, 1.0);
}

```

Shader Utility Header (`include/shader.h`)

```

#ifndef SHADER_H
#define SHADER_H

#include <string>
#include <glad/glad.h>
#include <glm/glm.hpp>

class Shader {
public:
    unsigned int ID;
    Shader(const char* vertexPath, const char* fragmentPath);
    void use() const;
    // uniform setters
    void setBool(const std::string &name, bool value) const;
    void setInt(const std::string &name, int value) const;
    void setFloat(const std::string &name, float value) const;
    void setVec3(const std::string &name, const glm::vec3
&value) const;
    void setMat4(const std::string &name, const glm::mat4 &mat)
const;
};

#endif

```

Shader Utility Implementation (src/shader.cpp)

```
#include "shader.h"
#include <fstream>
#include <sstream>
#include <iostream>

Shader::Shader(const char* vertexPath, const char* fragmentPath) {
    // 1. Retrieve shader source code from files
    std::string vertexCode, fragmentCode;
    std::ifstream vShaderFile(vertexPath), fShaderFile(fragmentPath);
    std::stringstream vStream, fStream;
    vStream << vShaderFile.rdbuf();
    fStream << fShaderFile.rdbuf();
    vertexCode = vStream.str();
    fragmentCode = fStream.str();
    vShaderFile.close();
    fShaderFile.close();

    // 2. Compile shaders
    const char* vCode = vertexCode.c_str();
    const char* fCode = fragmentCode.c_str();
    unsigned int vertex, fragment;
    int success;
    char infoLog[512];

    // Vertex
    vertex = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertex, 1, &vCode, nullptr);
    glCompileShader(vertex);
    glGetShaderiv(vertex, GL_COMPILE_STATUS, &success);
    if(!success) {
        glGetShaderInfoLog(vertex, 512, nullptr, infoLog);
        std::cerr << "ERROR: Vertex shader compilation failed\n" << infoLog
    << "\n";
    }

    // Fragment
    fragment = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragment, 1, &fCode, nullptr);
    glCompileShader(fragment);
    glGetShaderiv(fragment, GL_COMPILE_STATUS, &success);
    if(!success) {
        glGetShaderInfoLog(fragment, 512, nullptr, infoLog);
        std::cerr << "ERROR: Fragment shader compilation failed\n" << infoLog
    << "\n";
    }

    // Shader Program
    ID = glCreateProgram();
    glAttachShader(ID, vertex);
    glAttachShader(ID, fragment);
    glLinkProgram(ID);
    glGetProgramiv(ID, GL_LINK_STATUS, &success);
    if(!success) {
        glGetProgramInfoLog(ID, 512, nullptr, infoLog);
```

```

        std::cerr << "ERROR: Shader program linking failed\n" << infoLog <<
        "\n";
    }

    // Cleanup
    glDeleteShader(vertex);
    glDeleteShader(fragment);
}

void Shader::use() const {
    glUseProgram(ID);
}

void Shader::setBool(const std::string &name, bool value) const {
    glUniform1i(glGetUniformLocation(ID, name.c_str()), (int)value);
}

void Shader::setInt(const std::string &name, int value) const {
    glUniform1i(glGetUniformLocation(ID, name.c_str()), value);
}

void Shader::setFloat(const std::string &name, float value) const {
    glUniform1f(glGetUniformLocation(ID, name.c_str()), value);
}

void Shader::setVec3(const std::string &name, const glm::vec3 &value) const {
    glUniform3fv(glGetUniformLocation(ID, name.c_str()), 1, &value[0]);
}

void Shader::setMat4(const std::string &name, const glm::mat4 &mat) const {
    glUniformMatrix4fv(glGetUniformLocation(ID, name.c_str()), 1, GL_FALSE,
&mat[0][0]);
}

```

Main Application (src/main.cpp)

```

#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image.h>
#include <iostream>

#include "shader.h"

// callback for window resize
void framebuffer_size_callback(GLFWwindow* window, int width,
int height) {
    glViewport(0, 0, width, height);
}

int main() {
    // 1. Initialize GLFW
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);

```

```

    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE,
GLFW_OPENGL_CORE_PROFILE);

    // 2. Create window
    GLFWwindow* window = glfwCreateWindow(800, 600, "Textured
Cube", nullptr, nullptr);
    if (!window) { std::cerr << "Failed to create window\n";
return -1; }
    glfwMakeContextCurrent(window);
    glfwSetFramebufferSizeCallback(window,
framebuffer_size_callback);

    // 3. Load OpenGL functions via GLAD
    if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
        std::cerr << "Failed to initialize GLAD\n";
        return -1;
    }

    // 4. Build & compile our shader program
    Shader shader("shaders/basic.vert", "shaders/basic.frag");

    // 5. Set up vertex data (cube)
    float vertices[] = {
        // positions           // normals           // texcoords
        -0.5f,-0.5f,-0.5f,    0.0f,0.0f,-1.0f,    0.0f,0.0f,
        0.5f,-0.5f,-0.5f,    0.0f,0.0f,-1.0f,    1.0f,0.0f,
        0.5f, 0.5f,-0.5f,    0.0f,0.0f,-1.0f,    1.0f,1.0f,
        0.5f, 0.5f,-0.5f,    0.0f,0.0f,-1.0f,    1.0f,1.0f,
        -0.5f, 0.5f,-0.5f,    0.0f,0.0f,-1.0f,    0.0f,1.0f,
        -0.5f,-0.5f,-0.5f,    0.0f,0.0f,-1.0f,    0.0f,0.0f,
        // ... (repeat for all 6 faces, total 36 vertices)
    };

    unsigned int VAO, VBO;
    glGenVertexArrays(1, &VAO);
    glGenBuffers(1, &VBO);
    glBindVertexArray(VAO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

    // position attribute
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 *
sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
    // normal attribute

```

```

    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 *
sizeof(float), (void*)(3 * sizeof(float)));
    glEnableVertexAttribArray(1);
    // texture coord attribute
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 *
sizeof(float), (void*)(6 * sizeof(float)));
    glEnableVertexAttribArray(2);

    // 6. Load and create a texture
    unsigned int texture;
    glGenTextures(1, &texture);
    glBindTexture(GL_TEXTURE_2D, texture);
    // set texture wrapping/filtering
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_LINEAR);

    int width, height, nrChannels;
    unsigned char* data = stbi_load("assets/container.jpg",
&width, &height, &nrChannels, 0);
    if (data) {
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0,
GL_RGB, GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D);
    } else {
        std::cerr << "Failed to load texture\n";
    }
    stbi_image_free(data);

    // enable depth testing
    glEnable(GL_DEPTH_TEST);

    // 7. Render loop
    while (!glfwWindowShouldClose(window)) {
        // input
        if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
            glfwSetWindowShouldClose(window, true);

        // render
        glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    }

```

```

    // activate shader
    shader.use();
    shader.setVec3("lightPos", glm::vec3(1.2f, 1.0f, 2.0f));
    shader.setVec3("lightColor", glm::vec3(1.0f));
    shader.setVec3("viewPos", glm::vec3(0.0f, 0.0f, 3.0f));

    // create transformations
    glm::mat4 model = glm::mat4(1.0f);
    glm::mat4 view = glm::translate(glm::mat4(1.0f),
glm::vec3(0.0f, 0.0f, -3.0f));
    glm::mat4 projection =
glm::perspective(glm::radians(45.0f),
                                                         800.0f / 600.0f,
0.1f, 100.0f);
    shader.setMat4("model", model);
    shader.setMat4("view", view);
    shader.setMat4("projection", projection);

    // render the cube
    glBindVertexArray(VAO);
    glDrawArrays(GL_TRIANGLES, 0, 36);

    // swap buffers and poll IO
    glfwSwapBuffers(window);
    glfwPollEvents();
}

// cleanup
glDeleteVertexArrays(1, &VAO);
glDeleteBuffers(1, &VBO);
glfwTerminate();
return 0;
}

```